Python code, or the Python interpreter, using the following command:

```
bash>python
>>>import sys
>>>sys.path.append('/path/to/module')
```

Let's test the *hello_world()* function with the Python interpreter:

```
bash>python
>>>help(im)
Help on module InterfaceMethod:
NAME
    InterfaceMethod - Interface Method's docstring
FILE
    ...
FUNCTIONS
    ...
>>>im.hello_world.__doc__
'Prints Hello World!'
>>>im.hello_world()
Hello World!
```

You can see the public methods exposed by the module, and the documentation (doc-strings) using *help* and *__doc__*. Let's run *add_integers()* from a simple Python script given below:

```
bash>cat interface_add.py
import sys
import InterfaceMethod as im
def addIntegers(args=[]):
    a=0;b=0
    if len(args)>1 and args[1]:
        a=int(args[1])
    if len(args)>2 and  args[2]:
        b=int(args[2])
    if len(args)>3 and  args[3]:
        c=int(args[3])
        return im.add_integers(a,b,c)
    return im.add_integers(a,b)
def main(argv=None):
    response = addIntegers(sys.argv)
    print response
if __name__ == "__main__":
    sys.exit(main(sys.argv))
bash>python interface_add.py 2 3 5
sum = 10
10
```

Let's examine header decoding:

```
bash>cat interface_decode_hdr.py
#!/usr/bin/env python
import sys
import InterfaceMethod as im
def decodeHeaders(args=[]):
    if len(args) > 1 and args[1]:
        packet=args[1]
    else:
        packet="0x10234"
    return im.decode_header(packet)
def main(argv=None):
    res = decodeHeaders(sys.argv)
    if res[0]=="TRUE":
        print "Python:f1=%d f2=%d f3=%d f4=%d"
%(res[1],res[2],res[3],res[4])
    else:
        print "Failed to Decode"
if __name__ == "__main__":
    sys.exit(main(sys.argv))
bash>python interface_decode_hdr.py
 f1=4, f2=3 f3=2 f4=1
Python:f1=4 f2=3 f3=2 f4=1
```

Note: The above code is tested on an x86 machine. The header decoding output is dependent on the endianness of the system.

We have now finished coding and testing our first simple Python extension module! I hope you have enjoyed this article. END

Reference links

[1] http://docs.python.org/c-api/intro.html
[2] http://docs.python.org/c-api/arg.html
[3] http://docs.python.org/dev/c-api/refcounting.html
[4] http://docs.python.org/c-api/allocation.html
[5] http://docs.python.org/c-api/exceptions.html
[6] http://docs.python.org/distutils/introduction.html
[7] http://docs.python.org/install/index.html
[8] http://docs.python.org/c-api/list.html

Acknowledgement

I acknowledge everybody who contributed to Python, and the Python/C APIs, for inspiring this article. I also thank Sreekrishnan V for his support and tips.

By: Raghavendra K T

The author is a Linux enthusiast and a software developer. He has been working with IBM for the last 4.5 years, in the Embedded Systems Solutions Department. You can reach him at *raghavendra.kt@gmail.com*.